4.4 Further Examples of Recursion

In the remainder of this chapter, we provide additional examples of the use of recursion. We organize our presentation by considering the maximum number of recursive calls that may be started from within the body of a single activation.

- If a recursive call starts at most one other, we call this a ***linear recursion***.
- If a recursive call may start two others, we call this a ***binary recursion***.
- If a recursive call may start three or more others, this is ***multiple recursion***.

4.4.1 Linear Recursion

If a recursive function is designed so that each invocation of the body makes at most one new recursive call, this is know as ***linear recursion***. Of the recursions we have seen so far, the implementation of the factorial function (Section 4.1.1) and the good_fibonacci function (Section 4.3) are clear examples of linear recursion. More interestingly, the binary search algorithm (Section 4.1.3) is also an example of *linear recursion*, despite the "binary" terminology in the name. The code for binary search (Code Fragment 4.3) includes a case analysis with two branches that lead to recursive calls, but only one of those calls can be reached during a particular execution of the body.

A consequence of the definition of linear recursion is that any recursion trace will appear as a single sequence of calls, as we originally portrayed for the factorial function in Figure 4.1 of Section 4.1.1. Note that the *linear recursion* terminology reflects the structure of the recursion trace, not the asymptotic analysis of the running time; for example, we have seen that binary search runs in $O(\log n)$ time.

Summing the Elements of a Sequence Recursively

Linear recursion can be a useful tool for processing a data sequence, such as a Python list. Suppose, for example, that we want to compute the sum of a sequence, S, of n integers. We can solve this summation problem using linear recursion by observing that the sum of all n integers in S is trivially 0, if $n = 0$, and otherwise that it is the sum of the first $n-1$ integers in S plus the last element in S. (See Figure 4.9.)

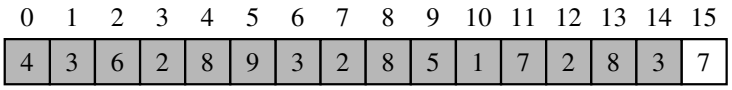


Figure 4.9: Computing the sum of a sequence recursively, by adding the last number to the sum of the first $n-1$.

A recursive algorithm for computing the sum of a sequence of numbers based on this intuition is implemented in Code Fragment 4.9.

```
1  def linear_sum(S, n):
2    """Return the sum of the first n numbers of sequence S."""
3    if n == 0:
4      return 0
5    else:
6      return linear_sum(S, n−1) + S[n−1]
```

Code Fragment 4.9: Summing the elements of a sequence using linear recursion.

A recursion trace of the linear_sum function for a small example is given in Figure 4.10. For an input of size n, the linear_sum algorithm makes $n+1$ function calls. Hence, it will take $O(n)$ time, because it spends a constant amount of time performing the nonrecursive part of each call. Moreover, we can also see that the memory space used by the algorithm (in addition to the sequence S) is also $O(n)$, as we use a constant amount of memory space for each of the $n+1$ activation records in the trace at the time we make the final recursive call (with $n = 0$).

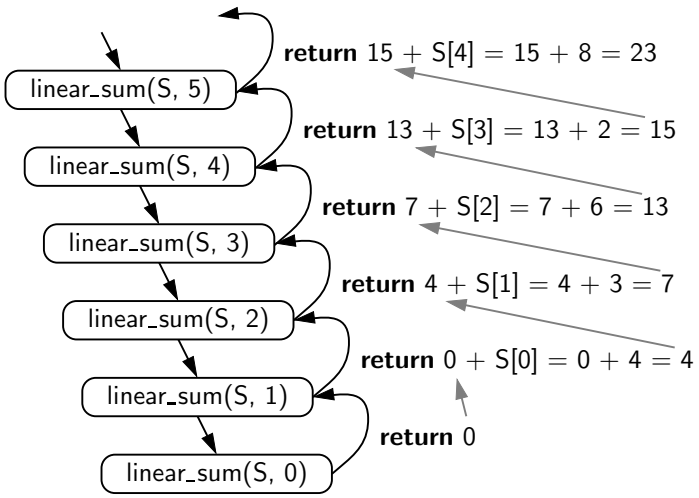


Figure 4.10: Recursion trace for an execution of linear_sum(S, 5) with input parameter S = [4, 3, 6, 2, 8].

Reversing a Sequence with Recursion

Next, let us consider the problem of reversing the n elements of a sequence, S, so that the first element becomes the last, the second element becomes second to the last, and so on. We can solve this problem using linear recursion, by observing that the reversal of a sequence can be achieved by swapping the first and last elements and then recursively reversing the remaining elements. We present an implementation of this algorithm in Code Fragment 4.10, using the convention that the first time we call this algorithm we do so as reverse(S, 0, len(S)).

```
1  def reverse(S, start, stop):
2    """Reverse elements in implicit slice S[start:stop]."""
3    if start < stop - 1:                    # if at least 2 elements:
4      S[start], S[stop-1] = S[stop-1], S[start]    # swap first and last
5      reverse(S, start+1, stop-1)                  # recur on rest
```

Code Fragment 4.10: Reversing the elements of a sequence using linear recursion.

Note that there are two implicit base case scenarios: When start == stop, the implicit range is empty, and when start == stop−1, the implicit range has only one element. In either of these cases, there is no need for action, as a sequence with zero elements or one element is trivially equal to its reversal. When otherwise invoking recursion, we are guaranteed to make progress towards a base case, as the difference, stop−start, decreases by two with each call (see Figure 4.11). If n is even, we will eventually reach the start == stop case, and if n is odd, we will eventually reach the start == stop − 1 case.

The above argument implies that the recursive algorithm of Code Fragment 4.10 is guaranteed to terminate after a total of $1 + \lfloor \frac{n}{2} \rfloor$ recursive calls. Since each call involves a constant amount of work, the entire process runs in $O(n)$ time.

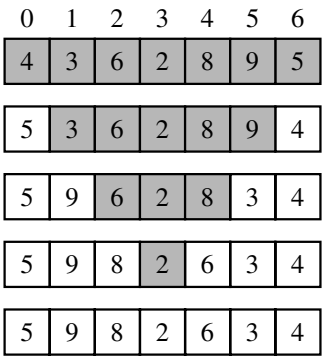


Figure 4.11: A trace of the recursion for reversing a sequence. The shaded portion has yet to be reversed.

Recursive Algorithms for Computing Powers

As another interesting example of the use of linear recursion, we consider the problem of raising a number x to an arbitrary nonnegative integer, n. That is, we wish to compute the ***power function***, defined as $power(x,n) = x^n$. (We use the name "power" for this discussion, to differentiate from the built-in function pow that provides such functionality.) We will consider two different recursive formulations for the problem that lead to algorithms with very different performance.

A trivial recursive definition follows from the fact that $x^n = x \cdot x^{n-1}$ for $n > 0$.

$$power(x,n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot power(x, n-1) & \text{otherwise.} \end{cases}$$

This definition leads to a recursive algorithm shown in Code Fragment 4.11.

```
1  def power(x, n):
2    """Compute the value x**n for integer n."""
3    if n == 0:
4      return 1
5    else:
6      return x * power(x, n-1)
```

Code Fragment 4.11: Computing the power function using trivial recursion.

A recursive call to this version of $power(x,n)$ runs in $O(n)$ time. Its recursion trace has structure very similar to that of the factorial function from Figure 4.1, with the parameter decreasing by one with each call, and constant work performed at each of $n+1$ levels.

However, there is a much faster way to compute the power function using an alternative definition that employs a squaring technique. Let $k = \lfloor \frac{n}{2} \rfloor$ denote the floor of the division (expressed as n // 2 in Python). We consider the expression $\left(x^k\right)^2$. When n is even, $\lfloor \frac{n}{2} \rfloor = \frac{n}{2}$ and therefore $\left(x^k\right)^2 = \left(x^{\frac{n}{2}}\right)^2 = x^n$. When n is odd, $\lfloor \frac{n}{2} \rfloor = \frac{n-1}{2}$ and $\left(x^k\right)^2 = x^{n-1}$, and therefore $x^n = x \cdot \left(x^k\right)^2$, just as $2^{13} = 2 \cdot 2^6 \cdot 2^6$. This analysis leads to the following recursive definition:

$$power(x,n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot \left(power\left(x, \lfloor \frac{n}{2} \rfloor\right)\right)^2 & \text{if } n > 0 \text{ is odd} \\ \left(power\left(x, \lfloor \frac{n}{2} \rfloor\right)\right)^2 & \text{if } n > 0 \text{ is even} \end{cases}$$

If we were to implement this recursion making *two* recursive calls to compute $power(x, \lfloor \frac{n}{2} \rfloor) \cdot power(x, \lfloor \frac{n}{2} \rfloor)$, a trace of the recursion would demonstrate $O(n)$ calls. We can perform significantly fewer operations by computing $power(x, \lfloor \frac{n}{2} \rfloor)$ as a partial result, and then multiplying it by itself. An implementation based on this recursive definition is given in Code Fragment 4.12.

```
1  def power(x, n):
2    """Compute the value x**n for integer n."""
3    if n == 0:
4      return 1
5    else:
6      partial = power(x, n // 2)          # rely on truncated division
7      result = partial * partial
8      if n % 2 == 1:                      # if n odd, include extra factor of x
9        result *= x
10     return result
```

Code Fragment 4.12: Computing the power function using repeated squaring.

To illustrate the execution of our improved algorithm, Figure 4.12 provides a recursion trace of the computation power(2, 13).

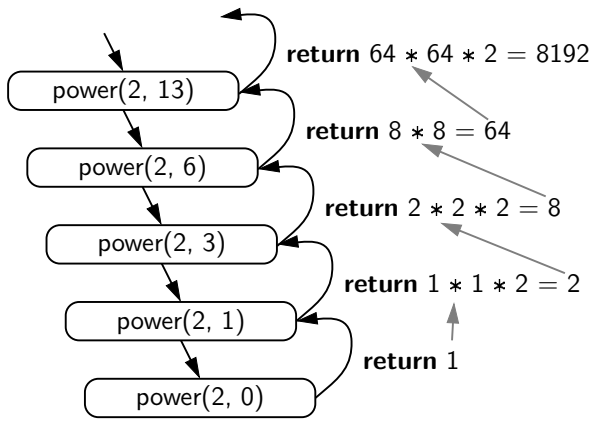


Figure 4.12: Recursion trace for an execution of power(2, 13).

To analyze the running time of the revised algorithm, we observe that the exponent in each recursive call of function power(x,n) is at most half of the preceding exponent. As we saw with the analysis of binary search, the number of times that we can divide n in half before getting to one or less is $O(\log n)$. Therefore, our new formulation of the power function results in $O(\log n)$ recursive calls. Each individual activation of the function uses $O(1)$ operations (excluding the recursive calls), and so the total number of operations for computing power(x,n) is $O(\log n)$. This is a significant improvement over the original $O(n)$-time algorithm.

The improved version also provides significant saving in reducing the memory usage. The first version has a recursive depth of $O(n)$, and therefore $O(n)$ activation records are simultaneous stored in memory. Because the recursive depth of the improved version is $O(\log n)$, its memory usages is $O(\log n)$ as well.

4.4.2 Binary Recursion

When a function makes two recursive calls, we say that it uses ***binary recursion***. We have already seen several examples of binary recursion, most notably when drawing the English ruler (Section 4.1.2), or in the bad_fibonacci function of Section 4.3. As another application of binary recursion, let us revisit the problem of summing the n elements of a sequence, S, of numbers. Computing the sum of one or zero elements is trivial. With two or more elements, we can recursively compute the sum of the first half, and the sum of the second half, and add these sums together. Our implementation of such an algorithm, in Code Fragment 4.13, is initially invoked as binary_sum(A, 0, len(A)).

```
1  def binary_sum(S, start, stop):
2    """Return the sum of the numbers in implicit slice S[start:stop]."""
3    if start >= stop:                          # zero elements in slice
4      return 0
5    elif start == stop-1:                      # one element in slice
6      return S[start]
7    else:                                      # two or more elements in slice
8      mid = (start + stop) // 2
9      return binary_sum(S, start, mid) + binary_sum(S, mid, stop)
```

Code Fragment 4.13: Summing the elements of a sequence using binary recursion.

To analyze algorithm binary_sum, we consider, for simplicity, the case where n is a power of two. Figure 4.13 shows the recursion trace of an execution of binary_sum(0, 8). We label each box with the values of parameters start:stop for that call. The size of the range is divided in half at each recursive call, and so the depth of the recursion is $1+\log_2 n$. Therefore, binary_sum uses $O(\log n)$ amount of additional space, which is a big improvement over the $O(n)$ space used by the linear_sum function of Code Fragment 4.9. However, the running time of binary_sum is $O(n)$, as there are $2n-1$ function calls, each requiring constant time.

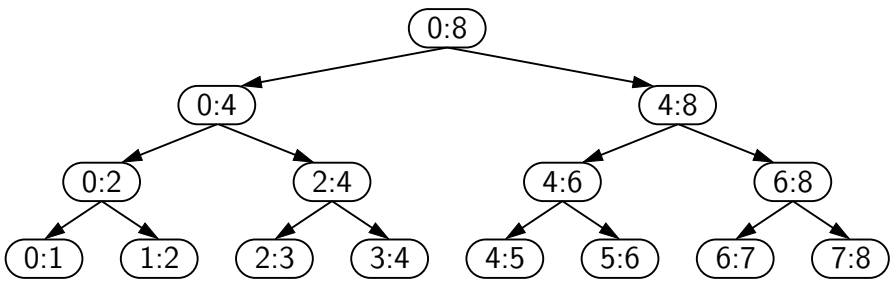


Figure 4.13: Recursion trace for the execution of binary_sum(0, 8).

4.4.3 Multiple Recursion

Generalizing from binary recursion, we define ***multiple recursion*** as a process in which a function may make more than two recursive calls. Our recursion for analyzing the disk space usage of a file system (see Section 4.1.4) is an example of multiple recursion, because the number of recursive calls made during one invocation was equal to the number of entries within a given directory of the file system.

Another common application of multiple recursion is when we want to enumerate various configurations in order to solve a combinatorial puzzle. For example, the following are all instances of what are known as ***summation puzzles***:

$$\begin{aligned} pot + pan &= bib \\ dog + cat &= pig \\ boy + girl &= baby \end{aligned}$$

To solve such a puzzle, we need to assign a unique digit (that is, $0, 1, \ldots, 9$) to each letter in the equation, in order to make the equation true. Typically, we solve such a puzzle by using our human observations of the particular puzzle we are trying to solve to eliminate configurations (that is, possible partial assignments of digits to letters) until we can work though the feasible configurations left, testing for the correctness of each one.

If the number of possible configurations is not too large, however, we can use a computer to simply enumerate all the possibilities and test each one, without employing any human observations. In addition, such an algorithm can use multiple recursion to work through the configurations in a systematic way. We show pseudocode for such an algorithm in Code Fragment 4.14. To keep the description general enough to be used with other puzzles, the algorithm enumerates and tests all k-length sequences without repetitions of the elements of a given universe U. We build the sequences of k elements by the following steps:

1. Recursively generating the sequences of $k-1$ elements
2. Appending to each such sequence an element not already contained in it.

Throughout the execution of the algorithm, we use a set U to keep track of the elements not contained in the current sequence, so that an element e has not been used yet if and only if e is in U.

Another way to look at the algorithm of Code Fragment 4.14 is that it enumerates every possible size-k ordered subset of U, and tests each subset for being a possible solution to our puzzle.

For summation puzzles, $U = \{0,1,2,3,4,5,6,7,8,9\}$ and each position in the sequence corresponds to a given letter. For example, the first position could stand for b, the second for o, the third for y, and so on.

```
Algorithm PuzzleSolve(k,S,U):
    Input: An integer k, sequence S, and set U
    Output: An enumeration of all k-length extensions to S using elements in U
      without repetitions
    for each e in U do
      Add e to the end of S
      Remove e from U                                   {e is now being used}
      if k == 1 then
        Test whether S is a configuration that solves the puzzle
        if S solves the puzzle then
          return "Solution found: " S
      else
        PuzzleSolve(k−1,S,U)                            {a recursive call}
      Remove e from the end of S
      Add e back to U                                   {e is now considered as unused}
```

Code Fragment 4.14: Solving a combinatorial puzzle by enumerating and testing all possible configurations.

In Figure 4.14, we show a recursion trace of a call to PuzzleSolve$(3,S,U)$, where S is empty and $U = \{a,b,c\}$. During the execution, all the permutations of the three characters are generated and tested. Note that the initial call makes three recursive calls, each of which in turn makes two more. If we had executed PuzzleSolve$(3,S,U)$ on a set U consisting of four elements, the initial call would have made four recursive calls, each of which would have a trace looking like the one in Figure 4.14.

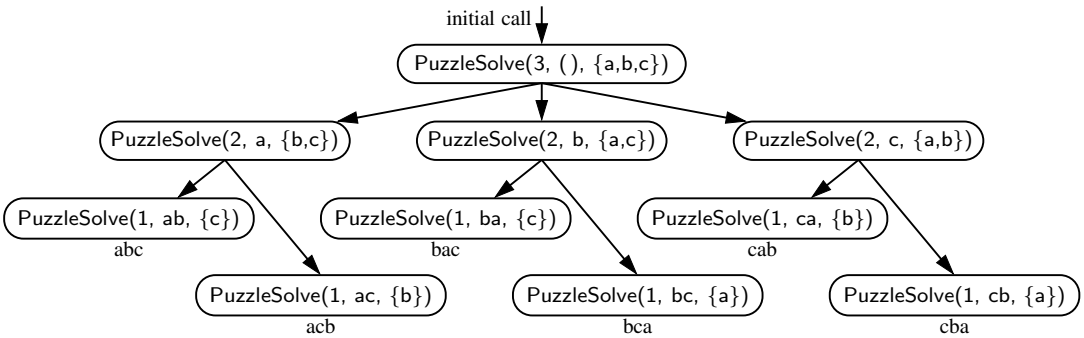


Figure 4.14: Recursion trace for an execution of PuzzleSolve$(3,S,U)$, where S is empty and $U = \{a,b,c\}$. This execution generates and tests all permutations of a, b, and c. We show the permutations generated directly below their respective boxes.